hw5

September 27, 2025

1 Assignment 5

Rex Wang

Submit a PDF or HTML export of your hw5.ipnyb file. Before exporting, select "Restart & Run All" to make sure all code runs cleanly and outputs are displayed. Also submit both .html plots.

1. Create a python file that webscrapes GDP by country. and plots a stacked interactive bar plot using plotly. Stack countries within regions using the IMF numbers. Please include this in your ipython notebook and output your plot to an html file containing the plot.

[39]:
```
import requests as rq
import bs4
import pandas as pd
from io import StringIO

# Webscrape the wikipedia page for GDP by country
url = 'https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)'
headers = {
    "User-Agent": "EN.585.771.81.FA25 HW5 (rwang183@jh.edu)"
}
page = rq.get(url, headers=headers)
bs4page = bs4.BeautifulSoup(page.text, 'html.parser')
tables = bs4page.find_all('table',{'class':"wikitable"})


# Read the table from the StringIO object into pandas
gdp = pd.read_html(StringIO(str(tables[0])))[0]
gdp = gdp.dropna()
gdp["Country/Territory"] = (
    gdp["Country/Territory"]
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                                # trim whitespace left over
)

gdpIMF = gdp[["Country/Territory", "IMF (2025)[1][6]"]].rename(columns={"IMF␣
 ↪(2025)[1][6]": "GDP (IMF)"})
gdpIMF = gdpIMF[gdpIMF["GDP (IMF)"] != "-"]
gdpIMF = gdpIMF.iloc[1:]
```

```
gdpIMF.head()
```

[39]:
```
  Country/Territory GDP (IMF)
1     United States  30507217
2             China  19231705
3           Germany   4744804
4             India   4187017
5             Japan   4186431
```

[40]:
```
# Webscrape the wikipedia page for country-continent mapping
urlContinent = 'https://simple.wikipedia.org/wiki/
 ↪List_of_countries_by_continents'
pageContinent = rq.get(urlContinent, headers=headers)
bs4pageContinent = bs4.BeautifulSoup(pageContinent.text, 'html.parser')
tablesContinent = bs4pageContinent.find_all('table',{'class':"wikitable"})

Africa = pd.read_html(StringIO(str(tablesContinent[0])))[0][["English Name␣
 ↪[1][2][3][4]"]].rename(columns={"English Name [1][2][3][4]": "Country/
 ↪Territory"})
Africa["Country/Territory"] = (
    Africa["Country/Territory"]
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                                # trim whitespace left over
)
Africa['Continent'] = 'Africa'
Africa.head()
```

[40]:
```
  Country/Territory Continent
0           Algeria    Africa
1            Angola    Africa
2             Benin    Africa
3          Botswana    Africa
4      Burkina Faso    Africa
```

[41]:
```
Asia = pd.read_html(StringIO(str(tablesContinent[2])))[0][["English Name␣
 ↪[4][10][11][12]"]].rename(columns={"English Name [4][10][11][12]": "Country/
 ↪Territory"})
Asia["Country/Territory"] = (
    Asia["Country/Territory"]
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                                # trim whitespace left over
)
Asia["Continent"] = 'Asia'
Asia.head()
```

[41]:
```
  Country/Territory Continent
0       Afghanistan      Asia
```

```
1           Armenia      Asia
2        Azerbaijan      Asia
3           Bahrain      Asia
4        Bangladesh      Asia
```

[42]:
```
Europe = pd.read_html(StringIO(str(tablesContinent[3])))[0]
Europe.columns = Europe.iloc[0]
Europe = Europe[1:][["English Name"]].rename(columns={"English Name": "Country/
↪Territory"})
Europe["Country/Territory"] = (
    Europe["Country/Territory"]
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                                # trim whitespace left over
)
Europe["Continent"] = 'Europe'
Europe.loc[3, "Country/Territory"] = "Austria"
Europe.head()
```

[42]:
```
0 Country/Territory Continent
1           Albania    Europe
2           Andorra    Europe
3           Austria    Europe
4           Belarus    Europe
5           Belgium    Europe
```

[43]:
```
N_America = pd.read_html(StringIO(str(tablesContinent[4])))[0][["English Name␣
↪[4][12][19][20]"]].rename(columns={"English Name [4][12][19][20]": "Country/
↪Territory"})
N_America["Country/Territory"] = (
    N_America["Country/Territory"]
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                                # trim whitespace left over
)
N_America["Continent"] = 'North America'
N_America.head()
```

[43]:
```
     Country/Territory      Continent
0  Antigua and Barbuda  North America
1          The Bahamas  North America
2             Barbados  North America
3               Belize  North America
4               Canada  North America
```

[44]:
```
S_America = pd.read_html(StringIO(str(tablesContinent[5])))[0][["English␣
↪Name"]].rename(columns={"English Name": "Country/Territory"})
S_America["Country/Territory"] = (
    S_America["Country/Territory"]
```

```
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                               # trim whitespace left over
)
S_America["Continent"] = 'South America'
S_America.head()
```

[44]:

	Country/Territory	Continent
0	Argentina	South America
1	Bolivia	South America
2	Brazil	South America
3	Chile	South America
4	Colombia	South America

[45]:
```
Oceania = pd.read_html(StringIO(str(tablesContinent[6])))[0][["English Name"]].
 ↪rename(columns={"English Name": "Country/Territory"})
Oceania["Country/Territory"] = (
    Oceania["Country/Territory"]
      .str.replace(r"\[.*?\]", "", regex=True)  # remove references
      .str.strip()                               # trim whitespace left over
)
Oceania["Continent"] = 'Oceania'
Oceania.head()
```

[45]:

	Country/Territory	Continent
0	Australia	Oceania
1	Cook Islands	Oceania
2	Federated States of Micronesia	Oceania
3	Fiji	Oceania
4	Kiribati	Oceania

[46]:
```
countryRegion = pd.concat([Africa, Asia, Europe, N_America, S_America,␣
 ↪Oceania], ignore_index=True)

# Merge the GDP data with the country-region mapping
gdp_region = pd.merge(gdpIMF, countryRegion, on='Country/Territory', how='left')


gdp_region.head()
```

[46]:

	Country/Territory	GDP (IMF)	Continent
0	United States	30507217	North America
1	China	19231705	Asia
2	Germany	4744804	Europe
3	India	4187017	Asia
4	Japan	4186431	Asia

[47]:
```
# fill in missing continents
gdp_region.loc[gdp_region['Country/Territory'] == 'Hong Kong', 'Continent'] =␣
 ↪'Asia'
gdp_region.loc[gdp_region['Country/Territory'] == 'Czech Republic',␣
 ↪'Continent'] = 'Europe'
gdp_region.loc[gdp_region['Country/Territory'] == 'Puerto Rico', 'Continent'] =␣
 ↪'North America'
gdp_region.loc[gdp_region['Country/Territory'] == 'Ivory Coast', 'Continent'] =␣
 ↪'Africa'
gdp_region.loc[gdp_region['Country/Territory'] == 'DR Congo', 'Continent'] =␣
 ↪'Africa'
gdp_region.loc[gdp_region['Country/Territory'] == 'Macau', 'Continent'] = 'Asia'
gdp_region.loc[gdp_region['Country/Territory'] == 'Bahamas', 'Continent'] =␣
 ↪'North America'
gdp_region.loc[gdp_region['Country/Territory'] == 'Aruba', 'Continent'] =␣
 ↪'South America'
gdp_region.loc[gdp_region['Country/Territory'] == 'Micronesia', 'Continent'] =␣
 ↪'Oceania'

# Check for any missing values in the merged DataFrame
print(gdp_region.isna().sum())
```

```
Country/Territory    0
GDP (IMF)            0
Continent            0
dtype: int64
```

[55]:
```
import plotly.express as px

gdp_region["GDP (IMF)"] = gdp_region["GDP (IMF)"].astype(float)
gdp_region["Continent"] = gdp_region["Continent"].astype('category')

fig = px.bar(
    gdp_region,
    x="Continent",
    y="GDP (IMF)",
    color="Country/Territory",
    title="GDP Distribution by Continent",
    barmode="stack",  # stack bars instead of grouping
    height=1000
)

fig.show()
```

[56]:
```
# Save to HTML
fig.write_html("stacked_bar.html", include_plotlyjs="cdn", full_html=True)
```

2. Look at the chapter on interactive graphics and, specifically, the code to display a subject's

MRICloud data as a sunburst plot. Do the following. Display this subject's data as a Sankey diagram. Display as many levels as you can (at least 3) for Type = 1, starting from the intracranial volume.

[50]:
```
## load in the hierarchy information
url = "https://raw.githubusercontent.com/bcaffo/MRIcloudT1volumetrics/master/
 ↪inst/extdata/multilevel_lookup_table.txt"
multilevel_lookup = pd.read_csv(url, sep = "\t").drop(['Level5'], axis = 1)
multilevel_lookup = multilevel_lookup.rename(columns = {
    "modify"   : "roi",
    "modify.1" : "level4",
    "modify.2" : "level3",
    "modify.3" : "level2",
    "modify.4" : "level1"})
multilevel_lookup = multilevel_lookup[['roi', 'level4', 'level3', 'level2',␣
 ↪'level1']]
multilevel_lookup.head()
```

[50]:

	roi	level4	level3	level2	level1
0	SFG_L	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L
1	SFG_R	SFG_R	Frontal_R	CerebralCortex_R	Telencephalon_R
2	SFG_PFC_L	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L
3	SFG_PFC_R	SFG_R	Frontal_R	CerebralCortex_R	Telencephalon_R
4	SFG_pole_L	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L

[51]:
```
import numpy as np

## load in the subject data
id = 127
subjectData = pd.read_csv("https://raw.githubusercontent.com/smart-stats/
 ↪ds4bio_book/main/book/assetts/kirby21AllLevels.csv")
subjectData = subjectData.loc[(subjectData.type == 1) & (subjectData.level ==␣
 ↪5) & (subjectData.id == id)]
subjectData = subjectData[['roi', 'volume']]
## Merge the subject data with the multilevel data
subjectData = pd.merge(subjectData, multilevel_lookup, on = "roi")
subjectData = subjectData.assign(icv = "ICV")
subjectData = subjectData.assign(comp = subjectData.volume / np.sum(subjectData.
 ↪volume))
subjectData.head()
```

[51]:

	roi	volume	level4	level3	level2	level1	\
0	SFG_L	12926	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L	
1	SFG_R	10050	SFG_R	Frontal_R	CerebralCortex_R	Telencephalon_R	
2	SFG_PFC_L	12783	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L	
3	SFG_PFC_R	11507	SFG_R	Frontal_R	CerebralCortex_R	Telencephalon_R	
4	SFG_pole_L	3078	SFG_L	Frontal_L	CerebralCortex_L	Telencephalon_L	

```
   icv      comp
0  ICV  0.009350
1  ICV  0.007270
2  ICV  0.009247
3  ICV  0.008324
4  ICV  0.002227
```

[52]:
```
import plotly.graph_objects as go

# Collect all unique labels
roi_label = subjectData['roi'].unique()
roi_label_to_id = {label: i for i, label in enumerate(roi_label)}

offset = len(roi_label)

level4_label = subjectData['level4'].unique()
level4_label_to_id = {label: (i+offset) for i, label in enumerate(level4_label)}

offset = offset + len(level4_label)

level3_label = subjectData['level3'].unique()
level3_label_to_id = {label: (i+offset) for i, label in enumerate(level3_label)}

offset = offset + len(level3_label)

level2_label = subjectData['level2'].unique()
level2_label_to_id = {label: (i+offset) for i, label in enumerate(level2_label)}

offset = offset + len(level2_label)

level1_label = subjectData['level1'].unique()
level1_label_to_id = {label: (i+offset) for i, label in enumerate(level1_label)}

# Build source-target-value links
source = []
target = []
value = []

# level 5 (roi) -> level 4
l514 = subjectData[['roi','volume','level4']]
l514 = l514.groupby(["roi","level4"], as_index=False)["volume"].sum()
for _,row in l514.iterrows():
    source.append(roi_label_to_id[row['roi']])
    target.append(level4_label_to_id[row['level4']])
    value.append(row['volume'])
```

```
# level 4 -> level 3
l4l3 = subjectData[['volume','level4','level3']]
l4l3 = l4l3.groupby(["level4","level3"], as_index=False)["volume"].sum()
for _,row in l4l3.iterrows():
    source.append(level4_label_to_id[row['level4']])
    target.append(level3_label_to_id[row['level3']])
    value.append(row['volume'])

# level 3 -> level 2
l3l2 = subjectData[['volume','level3','level2']]
l3l2 = l3l2.groupby(["level3","level2"], as_index=False)["volume"].sum()
for _,row in l3l2.iterrows():
    source.append(level3_label_to_id[row['level3']])
    target.append(level2_label_to_id[row['level2']])
    value.append(row['volume'])

# level 2 -> level 1
l2l1 = subjectData[['volume','level2','level1']]
l2l1 = l2l1.groupby(["level2","level1"], as_index=False)["volume"].sum()
for _,row in l2l1.iterrows():
    source.append(level2_label_to_id[row['level2']])
    target.append(level1_label_to_id[row['level1']])
    value.append(row['volume'])


# make figure
fig = go.Figure(data=[go.Sankey(
    valueformat = ".0f",
    valuesuffix = "TWh",
    # Define nodes
    node = dict(
      pad = 15,
      thickness = 15,
      line = dict(color = "black", width = 0.5),
      label =  np.concatenate([roi_label, level4_label, level3_label,␣
 ↪level2_label, level1_label]).tolist(),
    ),
    # Add links
    link = dict(
      source =  source,
      target =  target,
      value =  value
))])

fig.update_layout(
    title_text="ROI Volume Flow",
    font_size=12,
```

```
    height=5000)
fig.show()
```

[53]:
```
# Save to HTML
fig.write_html("sankey.html", include_plotlyjs="cdn", full_html=True)
```

3. Create a simple webpage containing the Sankey graphic and host it on github pages. Do not-host this off of your assignment repo from github classroom, since this is not public. Instead, you'll have to create a new public repo from your regular github account and add this file. Put the link to your live web page in a markdown cell of your hw5.ipynb file as a text block.

   https://sfyxboh.github.io/585.771-A5/sankey.html